TAGT Flare Predictor

SLE Flare Prediction using a Temporal Attention Graph Transformer

PROJECT PHASE – I REPORT

Submitted by

UTPAL SAKHIYA

VYOM PATEL

HARSHIL DHAMAT

POOJAN SHIHORA

Group: Token Burners

in partial fulfilment for the award of the degree of

BACHELOR OF TECHNOLOGY

In ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

Unitedworld Institute of Technology

Karnavati University

Uvarsad, Gandhinagar, Gujarat-382422

November, 2026

KARNAVATI UNIVERSITY
NAAC GRADE A+

UIT

TAGT Flare Predictor

SLE Flare Prediction using a Temporal Attention Graph Transformer

PROJECT PHASE – I REPORT

Submitted by

UTPAL SAKHIYA (202307020101)

VYOM PATEL (202307020084)

HARSHIL DHAMAT (202307020033)

POOJAN SHIHORA (202307020111)

Group: Token Burners

Under the guidance of

Prof. _____

in partial fulfilment for the award of the degree of

BACHELOR OF TECHNOLOGY

In ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

Unitedworld Institute of Technology
Karnavati University

Uvarsad, Gandhinagar, Gujarat-382422

November, 2026

KARNAVATI UNIVERSITY

Uvarsad, Gandhinagar, Gujarat-382422

Unitedworld Institute of Technology

BONAFIDE CERTIFICATE

Certified that this project report **"TAGT Flare Predictor: SLE Flare Prediction using a Temporal Attention Graph Transformer"** is the bonafide work of **"UTPAL SAKHIYA, VYOM PATEL, HARSHIL DHAMAT, POOJAN SHIHORA"**, members of the group "Token Burners", who carried out the Project Work (Phase-I) under my / our supervision.

SIGNATURE	**SIGNATURE**
HEAD OF THE DEPARTMENT	**SUPERVISOR**
Prof. _____	Prof. _____
Unitedworld Institute of Technology	Unitedworld Institute of Technology

Submitted for the project work viva-voce examination held on _____

INTERNAL EXAMINER	**EXTERNAL EXAMINER**

DECLARATION

We declare that this project report titled **"TAGT Flare Predictor: SLE Flare Prediction using a Temporal Attention Graph Transformer"** submitted in partial fulfilment of the degree of B. Tech in Artificial Intelligence and Machine Learning is a record of original work carried out by us under the supervision of **Prof. _____**, and has not formed the basis for the award of any other degree or diploma, in this or any other Institution or University. In keeping with the ethical practice in reporting scientific information, due acknowledgements have been made wherever the findings of others have been cited.

Name	**Enrollment No.**	**Signature**
Utpal Sakhiya	202307020101	_____
Vyom Patel	202307020084	_____
Harshil Dhamat	202307020033	_____
Poojan Shihora	202307020111	_____

ACKNOWLEDGEMENT

We express our sincere gratitude to our project supervisor, Prof. _____, for the guidance, encouragement and constructive feedback provided throughout this capstone project. The supervisor's insistence on careful validation directly shaped the evaluation methodology of this work.

We are thankful to the Head of the Department and to all faculty members of the Department of Artificial Intelligence and Machine Learning, Unitedworld Institute of Technology, Karnavati University, for providing the academic environment, computing resources and support that made this project possible.

We acknowledge the researchers who made the GSE49454 gene-expression dataset publicly available through the NCBI Gene Expression Omnibus, and the developers of the open-source tools used in this project, including PyTorch, scikit-learn, NumPy, pandas and Flask.

Finally, we thank our families and friends for their patience, motivation and constant support during the course of this work.

Utpal Sakhiya
Vyom Patel
Harshil Dhamat
Poojan Shihora

ABSTRACT

Systemic Lupus Erythematosus (SLE) is a chronic autoimmune disease in which periods of low activity are interrupted by flares that can cause irreversible organ damage. Anticipating a flare before the next clinical visit would allow earlier intervention. The objective of this project is to design, implement, validate and deploy a deep-learning system, the Temporal Attention Graph Transformer (TAGT), that estimates the probability of a flare at a patient's next visit from 1,000 whole-blood gene-expression probes and the current SLEDAI-2K disease-activity score.

The model combines a regularised gene encoder, a graph projection, multi-head attention and a clinical fusion branch. It was evaluated with patient-grouped stratified five-fold cross-validation, so every model is tested on unseen patients, and compared with three classical baselines on a semi-synthetic dataset of real GSE49454 expression profiles paired with simulated clinical visits. TAGT achieved an out-of-fold AUC-ROC of 0.598 (fold mean 0.609 ± 0.091), outperforming gene-based logistic regression (0.561) and random forest (0.529). A clinical-only logistic regression scored highest (0.669), showing that the current SLEDAI-2K score carries most of the signal in this data.

An audit showed that the previously distributed checkpoint was non-functional (AUC ≈ 0.50) and that an earlier claimed AUC of 0.943 is not reproducible without data leakage. The system is delivered as a Flask web application with a REST API, interactive predictions, gradient-based explanations and a transparent results dashboard.

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
3.1	System Architecture of the TAGT Flare Predictor	12
3.2	Architecture of the TAGT Network	13
4.1	Next-Visit Flare Rate and SLEDAI-2K Distribution by Outcome	16
4.2	Patient-Grouped Stratified 5-Fold Cross-Validation	18
5.1	Out-of-Fold ROC Curves of All Models	23
5.2	Pooled AUC with Fold Standard Deviation and Per-Fold AUC	24
5.3	Confusion Matrices of TAGT and the Clinical Baseline	25
5.4	Overview Page of the Web Application	27
5.5	Prediction and Explanation for a Dataset Visit	28
5.6	Prediction for a Custom Patient Profile	29
5.7	Results Page with ROC Curves, AUC Comparison and Metrics	30
5.8	Methodology Page	31
5.9	Team Page	31

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
2.1	Summary of Related Work	8
3.1	Hardware Requirements	10
3.2	Software Requirements	11
4.1	Summary of the Project Dataset	16
4.2	TAGT Training Hyper-parameters	17
4.3	REST API Endpoints	19
4.4	Evaluation of the Original Checkpoint under Different Input Conventions	21
4.5	Web Application Source Files	21
5.1	Out-of-Fold Performance under Patient-Grouped 5-Fold Cross-Validation	23
5.2	Confusion Matrix Counts at the Youden-Optimal Threshold	25
5.3	Effect of Regularisation Settings on TAGT (Out-of-Fold AUC)	26
5.4	Previously Claimed versus Reproduced Performance	32

LIST OF SYMBOLS, ABBREVIATIONS AND NOMENCLATURE

ABBREVIATIONS

ABBREVIATION	DESCRIPTION
API	Application Programming Interface
AUC	Area Under the (ROC) Curve
BCE	Binary Cross-Entropy
CPU	Central Processing Unit
CSV	Comma-Separated Values
CV	Cross-Validation
FN / FP	False Negative / False Positive
FPR	False Positive Rate
GAT	Graph Attention Network
GCN	Graph Convolutional Network
GEO	Gene Expression Omnibus (NCBI)
GNN	Graph Neural Network
GPU	Graphics Processing Unit
HTML / CSS / JS	HyperText Markup Language / Cascading Style Sheets / JavaScript
HTTP	HyperText Transfer Protocol
IFN	Interferon
JSON	JavaScript Object Notation
LR	Logistic Regression
LSTM	Long Short-Term Memory
MLP	Multi-Layer Perceptron

ABBREVIATION	DESCRIPTION
PPI	Protein–Protein Interaction
REST	Representational State Transfer
RF	Random Forest
ROC	Receiver Operating Characteristic
SD	Standard Deviation
SLE	Systemic Lupus Erythematosus
SLEDAI-2K	Systemic Lupus Erythematosus Disease Activity Index 2000
SVG	Scalable Vector Graphics
TAGT	Temporal Attention Graph Transformer
TN / TP	True Negative / True Positive
TPR	True Positive Rate (Sensitivity)
UI	User Interface

SYMBOLS AND NOMENCLATURE

SYMBOL	DESCRIPTION
x	Gene-expression vector of a visit (1,000 probes)
z	Standardised (z-scored) expression vector
s	Logit (pre-sigmoid output) of the TAGT network
c	Clinical feature vector [SLEDAI-2K / 20, current flare status]
μ, σ	Per-probe mean and standard deviation computed on the training fold
p	Predicted probability of a flare at the next visit
y	True label (1 = flare at next visit, 0 = no flare)
w_+	Positive-class weight in the loss (negatives / positives)
Q, K, V	Query, key and value matrices in attention

SYMBOL	DESCRIPTION
d_k	Dimension of each attention head
θ	Trainable model parameters
λ	Weight-decay coefficient
τ	Decision threshold on p
J	Youden's J statistic (sensitivity + specificity − 1)
a_i	Gradient × input attribution of probe i
$\sigma(\cdot)$	Logistic sigmoid function

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	DECLARATION	ii
	ACKNOWLEDGEMENT	iii
	ABSTRACT	iv
	LIST OF FIGURES	v
	LIST OF TABLES	vi
	LIST OF SYMBOLS, ABBREVIATIONS AND NOMENCLATURE	vii
1	**INTRODUCTION**	1
	1.1 GENERAL OVERVIEW	1
	1.2 BACKGROUND	1
	1.3 MOTIVATION	2
	1.4 PROBLEM STATEMENT	3
	1.5 OBJECTIVES	3
	1.6 SCOPE OF THE PROJECT	4
	1.7 ASSUMPTIONS, CONSTRAINTS AND ETHICAL CONSIDERATIONS	4
	1.8 ORGANISATION OF THE REPORT	4
2	**LITERATURE REVIEW**	5
	2.1 GENERAL OVERVIEW	5
	2.2 CLINICAL ASSESSMENT OF SLE ACTIVITY	5
	2.3 TRANSCRIPTOMIC SIGNATURES IN SLE	5
	2.4 MACHINE LEARNING FOR CLINICAL PREDICTION	6
	2.5 GRAPH NEURAL NETWORKS AND ATTENTION MECHANISMS	6
	2.6 VALIDATION PITFALLS: DATA LEAKAGE	7

CHAPTER NO.	TITLE	PAGE NO.
	2.7 EXPLAINABILITY OF DEEP MODELS	7
	2.8 COMPARATIVE SUMMARY	7
	2.9 RESEARCH GAP	8
	2.10 SUMMARY	9
3	**SYSTEM ARCHITECTURE AND DESIGN**	10
	3.1 GENERAL OVERVIEW	10
	3.2 SYSTEM REQUIREMENTS	10
	3.2.1 Hardware Requirements	10
	3.2.2 Software Requirements	10
	3.2.3 Functional Requirements	11
	3.2.4 Non-Functional Requirements	11
	3.3 SYSTEM ARCHITECTURE	12
	3.4 TAGT MODEL ARCHITECTURE	12
	3.4.1 Gene Encoder	13
	3.4.2 Graph Projection and Attention	13
	3.4.3 Clinical Fusion and Classifier	14
	3.5 DATA FLOW	14
	3.6 SUMMARY	14
4	**IMPLEMENTATION**	15
	4.1 DEVELOPMENT ENVIRONMENT	15
	4.2 DATASET DESCRIPTION	15
	4.2.1 Data Provenance	15
	4.2.2 Exploratory Analysis	16
	4.3 PREPROCESSING	16
	4.4 MODEL TRAINING	17
	4.5 VALIDATION PROTOCOL	18
	4.6 BASELINE MODELS	18

CHAPTER NO.	TITLE	PAGE NO.
	4.7 EXPLAINABILITY MODULE	19
	4.8 WEB APPLICATION	19
	4.8.1 Backend REST API	19
	4.8.2 Frontend User Interface	20
	4.8.3 Risk Banding Logic	20
	4.9 AUDIT OF THE ORIGINAL CHECKPOINT	20
	4.10 PROJECT STRUCTURE	21
	4.11 SUMMARY	21
5	**RESULTS AND DISCUSSION**	22
	5.1 GENERAL OVERVIEW	22
	5.2 EVALUATION METRICS	22
	5.3 CROSS-VALIDATION RESULTS	22
	5.4 ROC ANALYSIS	23
	5.5 FOLD-LEVEL STABILITY	24
	5.6 CONFUSION MATRIX ANALYSIS	24
	5.7 EFFECT OF REGULARISATION	25
	5.8 WEB APPLICATION RESULTS	26
	5.8.1 Overview Page	26
	5.8.2 Prediction for a Dataset Visit	27
	5.8.3 Prediction for a Custom Patient	28
	5.8.4 Results, Methodology and Team Pages	29
	5.9 COMPARISON WITH PREVIOUSLY REPORTED RESULTS	31
	5.10 DISCUSSION	32
	5.11 LIMITATIONS AND THREATS TO VALIDITY	33
	5.12 SUMMARY	33
6	**CONCLUSION AND FUTURE WORK**	34
	6.1 CONCLUSION	34

CHAPTER NO.	TITLE	PAGE NO.
	6.2 KEY OUTCOMES	34
	6.3 LIMITATIONS	34
	6.4 FUTURE WORK	35
	6.5 CLOSING REMARKS	35
	APPENDIX 1: INSTALLATION AND USER GUIDE	36
	APPENDIX 2: CORE SOURCE CODE	37
	REFERENCES	38

CHAPTER 1

INTRODUCTION

1.1 GENERAL OVERVIEW

Systemic Lupus Erythematosus (SLE) is a chronic, multi-system autoimmune disease in which the immune system produces auto-antibodies against the body's own tissues [24]. The disease follows a relapsing–remitting course: long periods of low activity are interrupted by **flares**, episodes of increased disease activity that may involve the skin, joints, kidneys, blood and nervous system [10]. Repeated flares accumulate organ damage, so the ability to anticipate a flare before it becomes clinically apparent is of great practical value.

This project, **TAGT Flare Predictor**, develops an end-to-end machine-learning system that estimates the probability that a patient will be in flare at the *next* clinical visit. It uses two complementary sources of information: a whole-blood gene-expression profile measured at the current visit (1,000 Illumina probes) and the patient's current clinical disease activity, expressed as the SLEDAI-2K score [7]. The predictive core is a **Temporal Attention Graph Transformer (TAGT)**, a neural network that combines a gene encoder, a graph projection, multi-head attention and a clinical fusion branch.

Beyond the model itself, the project delivers a complete software product: a reproducible training and validation pipeline, a Flask-based REST API and an interactive web interface. The interface lets a user run predictions, inspect which genes influenced a prediction and review the validated performance of every model. A central theme of the work is **honest validation**: all reported results are obtained with patient-grouped cross-validation, and earlier, unverified performance claims in the code base are audited and corrected.

1.2 BACKGROUND

SLE is diagnosed and classified using clinical and immunological criteria such as the SLICC classification criteria [17]. Once diagnosed, disease activity is monitored at regular

visits with validated indices. The most widely used is the SLEDAI-2K, a weighted sum of 24 clinical and laboratory descriptors ranging from 0 (no activity) upward; scores above roughly 6 indicate clinically significant activity [7]. Flare definitions are typically based on an increase in such indices together with the treating physician's assessment.

In parallel, transcriptomic profiling of whole blood has revealed reproducible molecular signatures of SLE. The most prominent is the **interferon (IFN) signature**, an over-expression of interferon-inducible genes found in a large proportion of patients [4]. Longitudinal studies of paediatric and adult cohorts have shown that modules of co-expressed genes track disease activity over time and can stratify patients into molecular subgroups [1]. The GSE49454 dataset used in this project originates from such a longitudinal study of adult SLE patients profiled on the Illumina HumanHT-12 platform [4, 13].

Modern deep-learning architectures offer tools suited to this kind of data. Attention mechanisms [25] learn which inputs to emphasise, and graph neural networks [11, 26] exploit known relationships between genes, such as protein–protein interaction networks [23]. The TAGT architecture explored in this project draws on both ideas.

1.3 MOTIVATION

Clinical decisions about SLE treatment are largely reactive: therapy is intensified after a flare has been recognised. A reliable early-warning signal could support closer monitoring of high-risk patients, earlier treatment adjustment and better allocation of clinic resources. Machine learning has shown promise across many areas of medicine [18], but published prediction models are often difficult to reproduce, rarely packaged as usable tools and frequently evaluated with protocols that overestimate their accuracy [5, 9].

The team was therefore motivated by three goals: to build a technically modern model for flare prediction, to evaluate it rigorously enough that its results can be trusted, and to package it as an accessible application that clinicians, researchers and students can use and understand.

1.4 PROBLEM STATEMENT

Given a patient's current-visit whole-blood gene-expression profile (1,000 probes), current SLEDAI-2K score and current flare status, predict the probability that the patient will be in flare at the next visit. Then deliver this prediction, together with an explanation and transparent validation evidence, through an easy-to-use web application.

The existing research code base addressed by this project suffered from several specific problems:

- **Unusable trained model:** the distributed checkpoint (`models/best_tagt_model.pt`) produced almost the same output (≈ 0.53) for every patient, giving chance-level discrimination.
- **Unverified performance claims:** project documents reported an AUC-ROC of 0.943, but the repository's own validation report marked these claims as not verified.
- **Risk of data leakage:** several visits come from the same patient, and record-wise cross-validation lets a patient's visits appear in both training and test folds, inflating scores [9].
- **No user-facing application:** the model could only be used by writing Python code, with no interface for prediction or interpretation.

1.5 OBJECTIVES

The objectives of the project are:

1. To study SLE flare prediction and the role of clinical activity scores and blood transcriptomics.
2. To implement the TAGT deep-learning architecture in PyTorch [15] and train it on the project dataset.
3. To evaluate TAGT and classical baselines with a leakage-free, patient-grouped stratified 5-fold cross-validation protocol.
4. To audit the previously distributed model checkpoint and the earlier performance claims.
5. To provide per-prediction explanations by attributing each prediction to individual gene probes.
6. To develop a Flask REST API and a responsive HTML/CSS/JavaScript web interface for prediction, explanation and results reporting.
7. To document the system, its validation and its limitations clearly and reproducibly.

1.6 SCOPE OF THE PROJECT

The project covers the complete pipeline from preprocessed data to a deployed web application: data loading and preprocessing, model design and training, cross-validated evaluation against baselines, model explainability, and a local web deployment that runs on an ordinary laptop without a GPU. The application accepts either a visit from the project dataset or a custom patient profile, including an optional uploaded expression file.

The project does **not** include prospective clinical data collection, external-cohort validation or regulatory approval. The system is a research and educational prototype and is not intended for clinical decision-making.

1.7 ASSUMPTIONS, CONSTRAINTS AND ETHICAL CONSIDERATIONS

- **Data:** the project dataset is semi-synthetic (Section 4.2). Expression profiles are real GSE49454 measurements, while visit trajectories, SLEDAI-2K scores and flare labels were simulated by the repository's preprocessing script. Conclusions about clinical utility therefore cannot be drawn.
- **Compute:** the system must train and run on a CPU-only machine in well under a minute, so model size and training length were kept modest.
- **Privacy:** GSE49454 is a public, de-identified dataset; no personally identifiable information is processed or stored by the application.
- **Responsible use:** the interface displays a research-prototype disclaimer, and all performance figures are reported with their uncertainty and limitations, following the spirit of the TRIPOD reporting guideline [5].

1.8 ORGANISATION OF THE REPORT

Chapter 2 reviews the literature on SLE activity assessment, transcriptomic signatures, graph and attention-based deep learning, data leakage and explainability. Chapter 3 presents the system requirements and architecture, including the TAGT model. Chapter 4 describes the implementation: dataset, preprocessing, training, validation, baselines, explainability and the web application. Chapter 5 reports and discusses the results, and Chapter 6 concludes the report and outlines future work. The appendices contain an installation guide and the core source code.

CHAPTER 2

LITERATURE REVIEW

2.1 GENERAL OVERVIEW

This chapter surveys the clinical, biological and computational work on which the project builds. It first summarises how SLE activity is measured clinically. It then reviews transcriptomic findings in SLE, machine-learning approaches to clinical prediction, and graph and attention-based neural networks. It closes with two methodological topics central to this project, validation pitfalls and model explainability, followed by a comparative summary and the identified research gap.

2.2 CLINICAL ASSESSMENT OF SLE ACTIVITY

SLE is heterogeneous in presentation, which makes both diagnosis and monitoring challenging [10, 24]. Classification criteria such as those of the Systemic Lupus International Collaborating Clinics combine clinical and immunological items to define the disease for research purposes [17]. For monitoring, the SLEDAI-2K index assigns weights to 24 descriptors, such as seizures, vasculitis, proteinuria, rash and low complement, observed in the preceding period [7]. Because the index is computed at each visit, a patient's SLEDAI-2K history forms a natural time series. An elevated current score is a well-known risk marker for continued or increasing activity, which makes it a strong baseline predictor for any flare model.

2.3 TRANSCRIPTOMIC SIGNATURES IN SLE

Microarray and sequencing studies of peripheral blood have characterised the molecular state of SLE patients. Chiche et al. analysed longitudinal blood transcriptomes of adult SLE patients (deposited in GEO as GSE49454) with a modular repertoire framework. They identified distinct type I and type II interferon signatures that were associated with disease activity [4]. Banchereau et al. followed paediatric patients longitudinally and showed that molecular networks, including interferon, plasmablast and neutrophil modules, stratify patients and correlate with activity over time [1]. These findings motivate the use of gene-

expression profiles as model inputs. They also indicate that any real predictive signal is likely to be spread across many co-regulated genes rather than concentrated in a single probe.

Public repositories such as the NCBI Gene Expression Omnibus [2] make these datasets available. Protein–protein interaction databases such as STRING [23] provide prior knowledge of gene relationships that graph-based models can exploit.

2.4 MACHINE LEARNING FOR CLINICAL PREDICTION

Classical machine-learning models remain strong baselines in clinical prediction, especially for small datasets. Logistic regression is interpretable and well calibrated. Random forests [3] capture non-linear interactions with little tuning. Both are available in scikit-learn [16]. Recurrent networks such as LSTMs [8] model sequences of visits, and deep networks can learn representations from high-dimensional omics data. Rajkomar et al. review the opportunities and pitfalls of machine learning in medicine. They emphasise that models must be evaluated in a way that reflects deployment conditions [18]. The TRIPOD statement provides a checklist for transparent reporting of such models, including the description of validation procedures and performance uncertainty [5].

2.5 GRAPH NEURAL NETWORKS AND ATTENTION MECHANISMS

Graph Convolutional Networks (GCNs) propagate information along graph edges through normalised neighbourhood aggregation [11]. Graph Attention Networks (GATs) replace fixed aggregation weights with learned attention coefficients, allowing each node to weight its neighbours differently [26]. The Transformer architecture introduced scaled dot-product multi-head attention as a general mechanism for relating elements of a set or sequence [25]. In genomics, these ideas are attractive because genes interact through pathways and regulatory networks. The TAGT architecture follows this line: it encodes the expression profile, projects it through a graph layer and applies multi-head attention before fusing it with clinical information.

2.6 VALIDATION PITFALLS: DATA LEAKAGE

Kaufman et al. formalise **leakage** as the introduction of information into model training that would not be available at prediction time, and show that it leads to overly optimistic performance estimates [9]. In longitudinal clinical data, a common form of leakage arises when several records from the same patient are split across training and test sets: the model can recognise the patient rather than learn generalisable patterns. The remedy is to split data by patient, for example with grouped cross-validation, where all records of a patient belong to exactly one fold. Scikit-learn's `StratifiedGroupKFold` implements this while also approximately preserving the class balance in each fold [19]. Feature scaling and selection must also be fitted on the training fold only.

2.7 EXPLAINABILITY OF DEEP MODELS

Clinical users need to understand why a model makes a prediction. Gradient-based saliency methods compute the derivative of the output with respect to each input feature [21]. Multiplying this gradient by the input value (gradient × input) gives an attribution that reflects both the sensitivity of the model to a feature and the magnitude of that feature [20]. These methods are cheap to compute, so they can be evaluated on demand in an interactive application.

2.8 COMPARATIVE SUMMARY

Table 2.1 summarises the key works reviewed and their relevance to this project.

Table 2.1 Summary of Related Work

Work	Contribution	Relevance to this project
Gladman et al. (2002)	SLEDAI-2K disease-activity index	Clinical input feature and strong baseline
Chiche et al. (2014)	Longitudinal blood transcriptomes of adult SLE (GSE49454); IFN signatures	Source of the expression profiles
Banchereau et al. (2016)	Molecular networks stratify lupus patients over time	Motivates multi-gene, network-aware models
Kipf & Welling (2017)	Graph Convolutional Networks	Graph projection concept in TAGT
Veličković et al. (2018)	Graph Attention Networks	Attention over gene relationships
Vaswani et al. (2017)	Multi-head scaled dot-product attention	Attention block of TAGT
Kaufman et al. (2012)	Formalisation of data leakage	Justifies patient-grouped validation
Collins et al. (2015)	TRIPOD reporting guideline	Transparent reporting of results
Shrikumar et al. (2017)	Gradient × input and related attribution methods	Per-prediction explanations

2.9 RESEARCH GAP

The literature establishes that clinical activity scores and blood transcriptomics both relate to SLE disease activity, and that graph and attention-based networks are suitable for modelling gene data. However, three gaps are evident in practice, and in the code base this project inherited:

- Reported deep-learning results are often not reproducible, and are evaluated without patient-level separation of training and test data.
- Complex models are rarely compared against simple clinical baselines, which can hide the fact that most of the signal comes from a single clinical variable.
- Research models are seldom delivered as usable, explainable applications for end users.

This project addresses these gaps by combining a TAGT model with a leakage-free validation protocol, direct comparison with clinical and genomic baselines, per-prediction explanations, and a deployable web application.

2.10 SUMMARY

This chapter reviewed the clinical measurement of SLE activity, the transcriptomic basis for using gene expression in flare prediction, and the deep-learning techniques underlying TAGT. It also covered the validation and explainability practices adopted in this work. The next chapter describes the system architecture designed to meet the project objectives.

CHAPTER 3

SYSTEM ARCHITECTURE AND DESIGN

3.1 GENERAL OVERVIEW

The TAGT Flare Predictor is organised as a three-layer system: a **data and model layer** that holds the dataset, the training pipeline and the trained artifacts; an **application layer** that exposes inference and explanation through a REST API; and a **presentation layer**, a browser-based user interface. This chapter specifies the system requirements and then describes the overall architecture and the design of the TAGT model.

3.2 SYSTEM REQUIREMENTS

3.2.1 Hardware Requirements

The system was deliberately designed to run on commodity hardware. Table 3.1 lists the minimum hardware requirements.

Table 3.1 Hardware Requirements

Component	Minimum Specification	Usage in Project
Processor	64-bit dual-core CPU (x86-64)	Model training and inference
Memory (RAM)	4 GB (8 GB recommended)	Holding the dataset and model in memory
Storage	2 GB free disk space	Python environment, dataset and artifacts
GPU	Not required	All training and inference run on CPU
Display / Network	Any display; network only for installation	Web interface served locally

3.2.2 Software Requirements

Table 3.2 lists the software stack. The versions shown are those with which the system was built and tested.

Table 3.2 Software Requirements

Software	Version	Purpose
Operating system	Windows 10/11 (Linux, macOS supported)	Host platform
Python	3.13 (3.10 or later)	Programming language
PyTorch (CPU)	2.15	TAGT model definition, training, gradients
scikit-learn	1.9	Baselines, cross-validation, metrics
NumPy / pandas	2.5 / 3.1	Numerical computing and data handling
Flask	3.1	REST API and static file server
joblib	1.x	Serialising the baseline model
Web browser	Any modern browser (Edge, Chrome, Firefox)	Running the user interface

3.2.3 Functional Requirements

- FR1: Train TAGT and the baseline models and evaluate them with patient-grouped 5-fold cross-validation.
- FR2: Persist the trained model, preprocessing statistics and metrics as reusable artifacts.
- FR3: Predict the next-visit flare probability for a selected dataset visit or for a custom patient profile.
- FR4: Accept an optional gene-expression CSV file and fill missing probes with the cohort mean.
- FR5: Explain each prediction by listing the ten probes with the largest attribution.
- FR6: Display validated performance (ROC curves, AUC comparison, full metrics table) and the project team.
- FR7: Validate user input and report errors clearly.

3.2.4 Non-Functional Requirements

- **Performance:** a prediction, including its explanation, is returned in well under one second on a CPU.
- **Portability:** the user interface uses no external libraries or CDNs and works offline once installed.
- **Usability:** responsive layout for desktop and mobile, light and dark themes, and plain-language explanations.
- **Reproducibility:** fixed random seeds and a single command (`python webapp/train.py`) regenerate all results.

- **Transparency:** every displayed metric is computed on unseen patients, and limitations are shown in the interface.

3.3 SYSTEM ARCHITECTURE

Fig 3.1 shows the overall architecture. In the data and model layer, the training pipeline (`train.py`) reads the dataset, runs cross-validation, refits the final models on all data and writes four artifacts: the TAGT weights, the preprocessing statistics, the clinical baseline model and a metrics file. In the application layer, the Flask server loads these artifacts once at start-up and exposes four REST endpoints. The inference engine standardises the inputs, runs TAGT and the baseline, and the explainability module computes probe attributions. In the presentation layer, the browser application calls the API and renders the results with lightweight SVG charts.

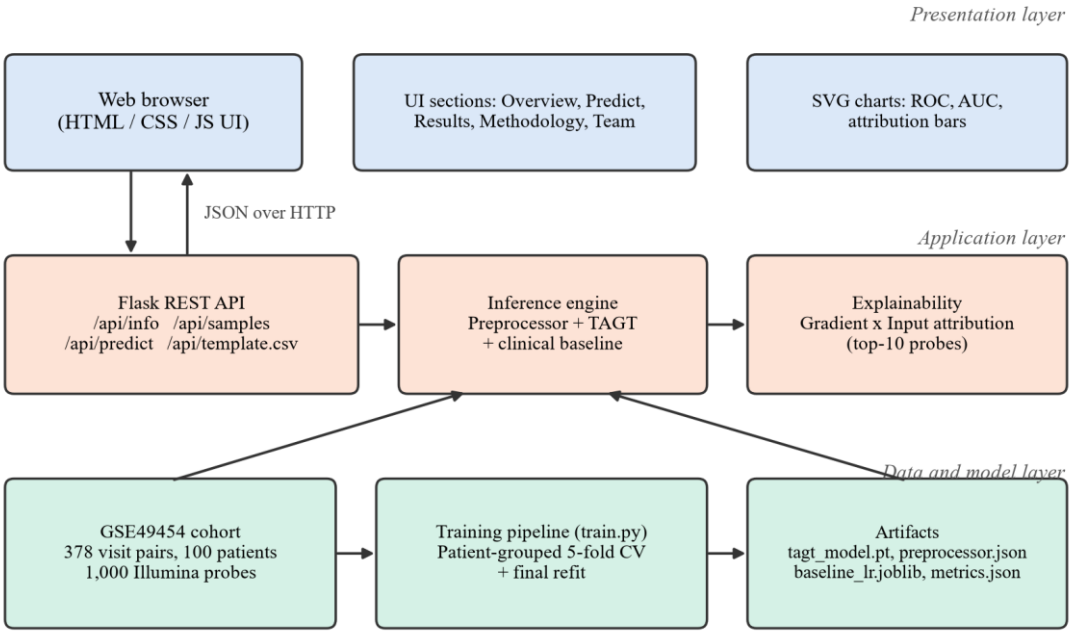


Fig 3.1 System Architecture of the TAGT Flare Predictor

3.4 TAGT MODEL ARCHITECTURE

The TAGT network (Fig 3.2) follows the layer layout of the original TAGT checkpoint in the code base: gene encoder, graph projection, attention, clinical encoder and classifier. It has 400,737 trainable parameters. It takes two inputs: the standardised expression vector **z**

(1,000 probes) and the clinical vector **c**, which contains the SLEDAI-2K score divided by 20 and the current flare status.

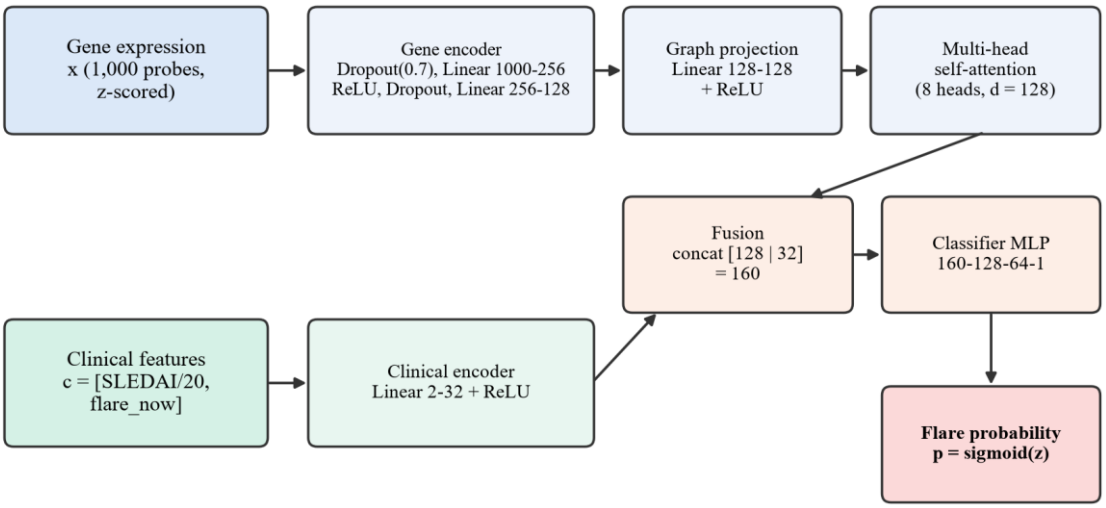


Fig 3.2 Architecture of the TAGT Network

3.4.1 Gene Encoder

The gene encoder compresses the 1,000-dimensional expression vector into a 128-dimensional molecular signature with a two-layer perceptron (1000 → 256 → 128) and ReLU activation. Because the dataset contains only 378 samples but 1,000 input features, the encoder begins with a **heavy input dropout** (p = 0.7) [22]. This randomly hides most probes during training, so the network cannot memorise individual samples. A second dropout layer (p = 0.3) follows the hidden layer.

3.4.2 Graph Projection and Attention

The molecular signature passes through a graph projection (a 128 × 128 linear map with ReLU) and then a multi-head self-attention block with eight heads [25]. Attention computes, for query, key and value matrices Q, K and V, the output shown in Eq. (3.1), where d_k = 16 is the per-head dimension. Each head operates on a different learned projection and the head outputs are concatenated, as in Eq. (3.2).

$$Attention(Q, K, V) = softmax(Q K^T / \sqrt{d_k}) V \quad (3.1)$$

$$MultiHead(Q, K, V) = Concat(head_1, \ldots, head_8) W^O \quad (3.2)$$

In the deployed configuration each visit is represented by a single time step, so the attention block acts as a learned re-weighting of the gene signature. The same interface accepts sequences of several visits, which keeps the design open for future longitudinal extensions (Section 6.4).

3.4.3 Clinical Fusion and Classifier

The clinical vector **c** is encoded into 32 features (Linear 2 → 32, ReLU) and concatenated with the 128-dimensional attention output into a 160-dimensional fused representation. A three-layer classifier (160 → 128 → 64 → 1, ReLU, dropout 0.3) produces a logit *s*, which is mapped to a flare probability with the sigmoid function, as in Eq. (3.3).

$$p = \sigma(s) = 1 / (1 + e^{-s}) \tag{3.3}$$

3.5 DATA FLOW

The end-to-end data flow for a single prediction request is:

1. The user selects a dataset visit, or enters a SLEDAI-2K score, current flare status and an optional expression CSV, and presses Predict.
2. The browser sends a JSON request to `POST /api/predict`.
3. The server builds the expression vector (missing probes are filled with the cohort mean), standardises it with the stored training statistics and scales the clinical features.
4. TAGT computes the flare probability; the clinical baseline computes its own probability for comparison.
5. The gradient of the TAGT output with respect to the input is computed and multiplied by the input to obtain probe attributions.
6. The server returns probabilities, risk bands, decision thresholds and the top-10 attributions as JSON; the browser renders them.

3.6 SUMMARY

This chapter defined the hardware, software, functional and non-functional requirements, and presented the layered system architecture and the TAGT network design. The next chapter explains how each component was implemented.

CHAPTER 4

IMPLEMENTATION

4.1 DEVELOPMENT ENVIRONMENT

The system was implemented in Python 3.13 inside an isolated virtual environment (`.venv`). PyTorch [15] was used for the neural network and automatic differentiation, scikit-learn [16] for baselines, cross-validation and metrics, and Flask [14] for the web server. The user interface is written in plain HTML, CSS and JavaScript without third-party libraries. A Windows batch script, `run_webapp.bat`, creates the environment, installs the dependencies and starts the server in one step.

4.2 DATASET DESCRIPTION

4.2.1 Data Provenance

The expression data come from the public GEO series **GSE49454**, which contains whole-blood transcriptomes of adult SLE patients measured on the Illumina HumanHT-12 platform [4, 13]. The repository's preprocessing script (`src/data/process_real_data.py`) retained 1,000 probes from 177 arrays and then constructed the longitudinal dataset used for training.

An inspection of this script during the project showed that the clinical part of the dataset is **simulated**. The script generates 100 virtual patients with 2–8 visits each and random SLEDAI-2K trajectories, and draws flare events with a probability that increases with the SLEDAI-2K score. It then assigns each visit a *randomly chosen* real expression profile with small added noise. Consecutive visits are paired, and the label is the flare status at the second visit. The resulting dataset is therefore **semi-synthetic**: the expression values are real, but their link to the clinical outcome is artificial. This fact is essential for interpreting the results in Chapter 5, and it is stated in the web interface. Table 4.1 summarises the dataset.

Table 4.1 Summary of the Project Dataset

Property	Value
Expression source	GSE49454, Illumina HumanHT-12 (177 arrays)
Probes used	1,000 ($\log_2$ expression, range 1.27–14.27)
Patients (simulated)	100
Consecutive-visit pairs	378 (1–7 per patient, mean 3.78)
Positive class (flare at next visit)	128 pairs (33.9%)
Flaring at current visit	32.3% of pairs
Current SLEDAI-2K	mean 13.6, SD 8.6, range 0–42.6

4.2.2 Exploratory Analysis

Fig 4.1 shows the relationship between the current SLEDAI-2K score and the next-visit outcome. The flare rate rises steadily with disease activity, from 21% for scores of 0–4 to 60% for scores of 20 or more. The two outcome distributions overlap substantially, however, so SLEDAI-2K alone cannot separate the classes perfectly.

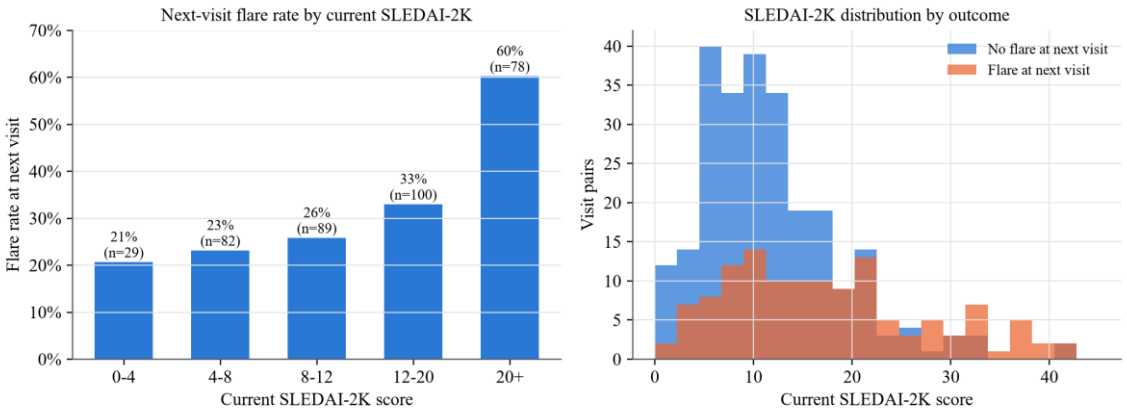


Fig 4.1 Next-Visit Flare Rate and SLEDAI-2K Distribution by Outcome

4.3 PREPROCESSING

Each probe is standardised to zero mean and unit variance using statistics computed **only on the training fold**, as in Eq. (4.1). Here x_i is the $\log_2$ expression of probe i, and μ_i and σ_i are its training-fold mean and standard deviation (a small constant is added to σ_i to avoid division by zero). The SLEDAI-2K score is divided by 20 so that it lies on a scale

comparable to the other inputs, and the current flare status is used as a 0/1 indicator. For deployment, the statistics computed on the full dataset are saved in `preprocessor.json`.

$$z_i = (x_i - \mu_i) / \sigma_i \tag{4.1}$$

4.4 MODEL TRAINING

TAGT is trained by minimising a class-weighted binary cross-entropy loss, given in Eq. (4.2). Here N is the batch size and the positive-class weight w_+ equals the ratio of negative to positive training samples, about 1.95, which compensates for the class imbalance. Optimisation uses AdamW [12], which applies decoupled weight decay λ to the parameters θ.

$$L(\theta) = -(1/N) \Sigma [\, w_+ \, y \log p + (1 - y) \log(1 - p) \,] \tag{4.2}$$

To make the network robust to measurement noise, Gaussian noise with standard deviation 0.1 is added to the standardised expression of each training batch. Gradients are clipped to a norm of 1.0. Table 4.2 lists the final hyper-parameters. Training the five cross-validation models and the final model takes about 17 seconds on a laptop CPU.

Table 4.2 TAGT Training Hyper-parameters

Hyper-parameter	Value
Optimiser	AdamW
Learning rate	1×10^{-3}
Weight decay (λ)	0.05
Epochs	15
Batch size	32
Input dropout / hidden dropout	0.7 / 0.3
Attention heads	8 (model dimension 128)
Noise augmentation (σ)	0.1
Gradient clipping (max norm)	1.0
Loss	Class-weighted BCE (w_+ = negatives / positives)
Random seed	42 (+ fold index)

4.5 VALIDATION PROTOCOL

All models are evaluated with **patient-grouped stratified 5-fold cross-validation** (`StratifiedGroupKFold`, seed 42) [19]. As Fig 4.2 illustrates, the 100 patients are divided into five groups of about 20. In each round, one group is held out for testing, and all visits of a held-out patient are excluded from training. Preprocessing statistics are fitted on the training folds only. Predictions on the five test folds are collected into a single set of **out-of-fold** predictions covering every sample exactly once, from which pooled metrics are computed. The spread of the per-fold AUC values is also reported as a measure of stability.

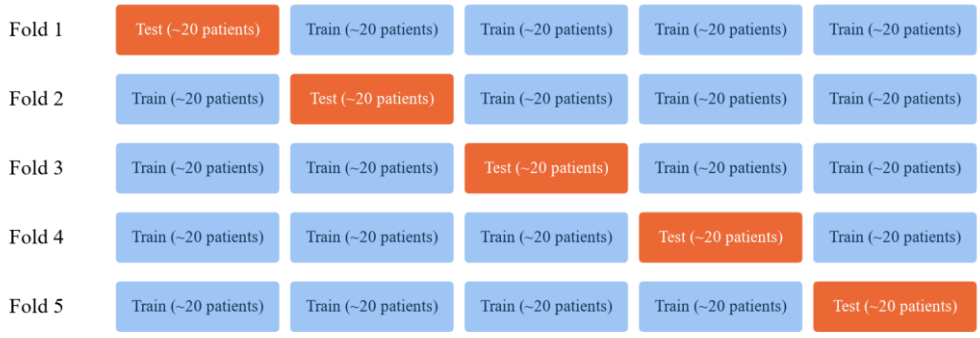


Fig 4.2 Patient-Grouped Stratified 5-Fold Cross-Validation

Binary predictions require a decision threshold τ. For each model, τ is chosen on the out-of-fold predictions to maximise Youden's J statistic [27], defined in Eq. (4.3).

$$J(\tau) = Sensitivity(\tau) + Specificity(\tau) - 1 \tag{4.3}$$

4.6 BASELINE MODELS

Three baselines were trained under exactly the same folds:

- **Logistic Regression (clinical):** standardised SLEDAI-2K and current flare status only. This measures how much can be achieved without any gene data.
- **Logistic Regression (genes + clinical):** all 1,000 probes plus the clinical features, with strong L2 regularisation (C = 0.01).
- **Random Forest (genes + clinical):** 300 trees with a minimum of three samples per leaf [3].

After evaluation, the clinical logistic regression is refitted on all data and served by the web application alongside TAGT as a reference point for each prediction.

4.7 EXPLAINABILITY MODULE

For every prediction, the server computes the gradient of the TAGT logit *s* with respect to the standardised expression vector using PyTorch's automatic differentiation. It then multiplies the gradient element-wise by the input, giving the attribution in Eq. (4.4) [20, 21]. A positive a_i indicates that probe i pushed the prediction towards a flare, and a negative value indicates the opposite. The ten probes with the largest $|a_i|$ are returned with their expression values and z-scores. When no expression file is supplied, all probes sit at the cohort mean (z = 0), so all attributions are zero. The interface then explains that the prediction is driven by the clinical inputs alone.

$$a_i = z_i \cdot \partial s / \partial z_i \tag{4.4}$$

4.8 WEB APPLICATION

4.8.1 Backend REST API

The backend (`webapp/app.py`) is a Flask application [14]. At start-up it checks for the trained artifacts and runs the training pipeline automatically if any are missing. It then loads the dataset, model, preprocessing statistics and baseline into memory. Table 4.3 lists the endpoints. Inputs are validated: out-of-range sample indices and SLEDAI-2K values, or non-numeric fields, return HTTP 400 with a descriptive error message.

Table 4.3 REST API Endpoints

Method	Endpoint	Description
GET	/	Serves the single-page web interface
GET	/api/info	Project and team details and all cross-validation metrics
GET	/api/samples	List of the 378 dataset visit pairs with clinical values and outcomes
POST	/api/predict	Prediction for {sample_index} or {current_sledai, current_flare, expression}
GET	/api/template.csv	Downloadable expression template (1,000 probes, cohort means)

4.8.2 Frontend User Interface

The frontend is a single-page application made of three static files (`index.html`, `style.css`, `app.js`) with hash-based navigation between five sections: Overview, Predict, Results, Methodology and Team. Charts are drawn as inline SVG: ROC curves with a hover crosshair, an AUC bar chart with fold variance whiskers, and diverging bars for probe attributions. Avoiding chart libraries keeps the application fully functional offline. The layout is responsive down to mobile widths, and colour tokens provide light and dark themes, with colour-blind-safe series colours and text labels so that no information is conveyed by colour alone.

4.8.3 Risk Banding Logic

Each probability is reported with a risk band relative to the model's decision threshold τ. The band is **High** when $p \geq \tau$, so a flare is predicted. It is **Moderate** when $\tau - 0.10 \leq p < \tau$, a borderline case below the threshold, and **Low** otherwise. The thresholds are $\tau = 0.59$ for TAGT and $\tau = 0.39$ for the clinical baseline, chosen by Youden's J on the out-of-fold predictions.

4.9 AUDIT OF THE ORIGINAL CHECKPOINT

Before retraining, the checkpoint shipped with the code base (`models/best_tagt_model.pt`, 1.6 MB) was reconstructed and evaluated on the project dataset. The original preprocessing was not documented, so several plausible input conventions were tested (Table 4.4). Under every convention the model produced almost identical outputs for all samples, and its AUC was at or below chance. The checkpoint was therefore kept unchanged for reference but not used. A new model was trained as described above and saved separately as `webapp/artifacts/tagt_model.pt`.

Table 4.4 Evaluation of the Original Checkpoint under Different Input Conventions

Expression input	Clinical input	AUC-ROC	Output range
Raw $\log_2$	Zero	0.496	0.53 – 0.54
Raw $\log_2$	SLEDAI-2K / 20	0.337	0.49 – 0.53
Standardised	Zero	0.507	0.52 – 0.53
Standardised	SLEDAI-2K (raw)	0.327	0.13 – 0.53
Standardised	Standardised SLEDAI-2K	0.353	0.47 – 0.53

4.10 PROJECT STRUCTURE

Table 4.5 lists the files added to the code base for the deployable system.

Table 4.5 Web Application Source Files

File	Responsibility
webapp/tagt_core.py	TAGT architecture, dataset loading, preprocessing
webapp/train.py	Cross-validation, baselines, final refit, metrics export
webapp/app.py	Flask server, REST API, inference and explanations
webapp/static/index.html	Page structure for the five UI sections
webapp/static/style.css	Responsive layout, light/dark theme tokens
webapp/static/app.js	API calls, prediction flow, SVG charts, tooltips
webapp/artifacts/	tagt_model.pt, preprocessor.json, baseline_lr.joblib, metrics.json
run_webapp.bat	One-click setup and launch on Windows
requirements-webapp.txt	Python dependencies of the web application
README_WEBAPP.md	Installation, API and validity notes

4.11 SUMMARY

This chapter described the dataset and its semi-synthetic provenance, the preprocessing, the training and validation procedures, the baselines, the explainability module and the web application. It also documented the audit of the original checkpoint. The results obtained with this implementation are presented next.

CHAPTER 5

RESULTS AND DISCUSSION

5.1 GENERAL OVERVIEW

This chapter reports the cross-validated performance of TAGT and the three baselines, analyses their ROC curves, fold-level stability and error patterns, and presents the deployed web application. It then compares the results with the performance claimed earlier in the code base and discusses their meaning and limitations.

5.2 EVALUATION METRICS

The primary metric is the area under the ROC curve (AUC-ROC). It measures how well a model ranks flare cases above non-flare cases independently of any threshold: 0.5 corresponds to random ranking and 1.0 to perfect ranking [6]. At the Youden-optimal threshold, the confusion-matrix counts TP, TN, FP and FN give the following metrics:

$$Sensitivity = TP / (TP + FN), \quad Specificity = TN / (TN + FP) \tag{5.1}$$

$$Accuracy = (TP + TN) / (TP + TN + FP + FN) \tag{5.2}$$

$$F1 = 2TP / (2TP + FP + FN) \tag{5.3}$$

5.3 CROSS-VALIDATION RESULTS

Table 5.1 reports the out-of-fold results of all four models. TAGT achieves a pooled AUC-ROC of **0.598** (fold mean 0.609 ± 0.091) and clearly outperforms both models that use the 1,000 gene probes directly: L2-regularised logistic regression (0.561) and random forest (0.529). The clinical-only logistic regression attains the highest AUC (**0.669**), with the lowest fold variance (± 0.043).

Table 5.1 Out-of-Fold Performance under Patient-Grouped 5-Fold Cross-Validation

Model	AUC	Fold AUC (mean ± SD)	Accuracy	Sensitivity	Specificity	F1
TAGT (ours)	**0.598**	**0.609 ± 0.091**	**65.9%**	**42.2%**	**78.0%**	**0.456**
LogReg (clinical)	0.669	0.669 ± 0.043	70.1%	48.4%	81.2%	0.523
LogReg (genes + clinical)	0.561	0.573 ± 0.074	59.5%	49.2%	64.8%	0.452
Random Forest (genes + clinical)	0.529	0.535 ± 0.063	58.7%	37.5%	69.6%	0.381

5.4 ROC ANALYSIS

Fig 5.1 shows the pooled out-of-fold ROC curves. The clinical logistic regression dominates in the low false-positive region, which is the most relevant region for an early-warning tool. TAGT lies above both gene-based baselines over most of the range, and all curves approach the chance diagonal at high false-positive rates.

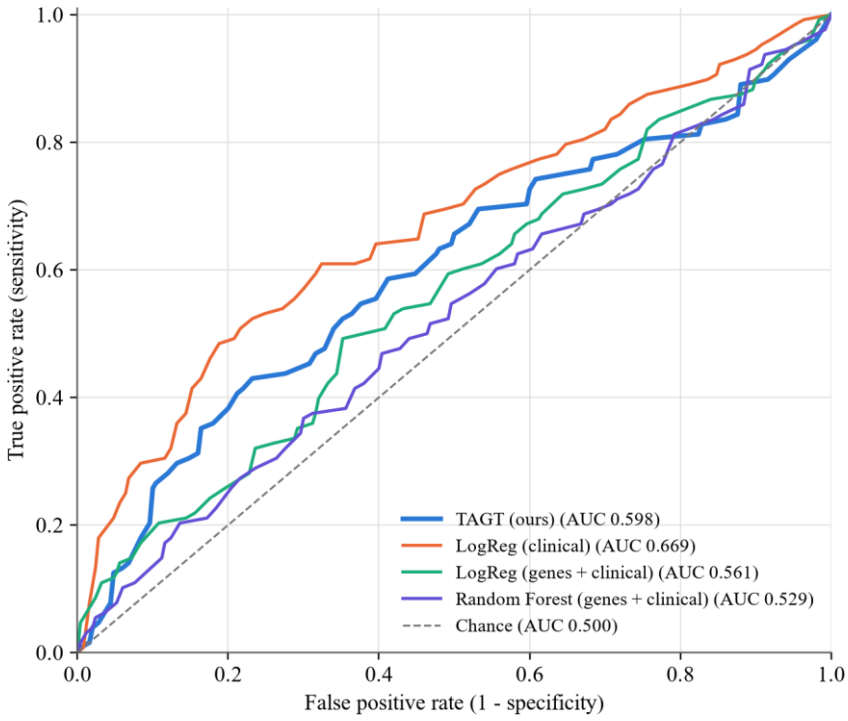


Fig 5.1 Out-of-Fold ROC Curves of All Models

5.5 FOLD-LEVEL STABILITY

Fig 5.2 compares the pooled AUCs (left) and the AUC of each fold (right). TAGT reaches 0.713 on fold 1, the best single-fold result of any model, but drops to 0.446 on fold 4. Each test fold contains only about 20 patients and 75 visit pairs, so a few patients can move the AUC considerably. The clinical model is the most stable. Fold 4 is difficult for every gene-based model, which suggests that the patients in that fold are atypical with respect to the expression features.

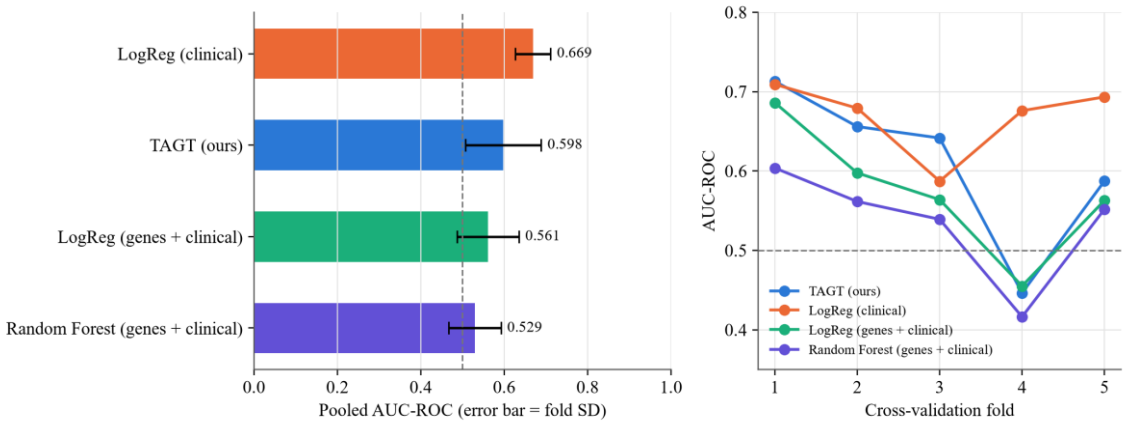


Fig 5.2 Pooled AUC with Fold Standard Deviation and Per-Fold AUC

5.6 CONFUSION MATRIX ANALYSIS

Fig 5.3 shows the confusion matrices of TAGT and the clinical baseline at their Youden thresholds, and Table 5.2 lists the counts. Both models favour specificity over sensitivity. TAGT correctly identifies 54 of the 128 flares while raising 55 false alarms among 250 non-flare visits. The clinical model identifies 62 flares with 47 false alarms.

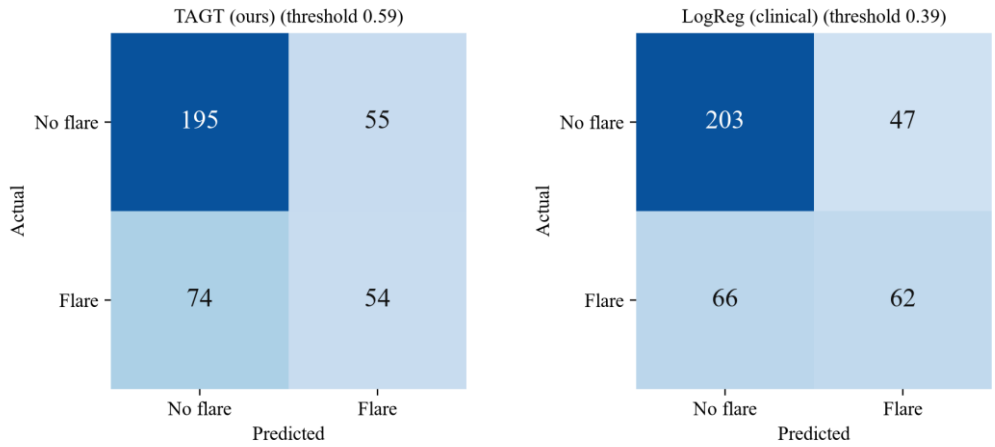


Fig 5.3 Confusion Matrices of TAGT and the Clinical Baseline

Table 5.2 Confusion Matrix Counts at the Youden-Optimal Threshold

Model	Threshold	TP	FN	FP	TN
TAGT (ours)	0.59	54	74	55	195
LogReg (clinical)	0.39	62	66	47	203
LogReg (genes + clinical)	0.35	63	65	88	162
Random Forest (genes + clinical)	0.43	48	80	76	174

5.7 EFFECT OF REGULARISATION

During development, several regularisation settings of TAGT were compared on the same folds (Table 5.3). Without input dropout the network overfits the 1,000 probes and reaches only 0.538. Heavy input dropout with stronger weight decay raised the AUC to 0.598. A variant with an additional direct clinical-to-output skip connection did not help. These settings were selected on the same cross-validation folds that are used for reporting, so the selected configuration's AUC is slightly optimistic. This is acknowledged as a threat to validity in Section 5.11.

Table 5.3 Effect of Regularisation Settings on TAGT (Out-of-Fold AUC)

Input dropout	Epochs	Weight decay	Variant	AUC-ROC
0.0	40	0.01	Base	0.538
0.5	15	0.01	Base	0.553
0.5	8	0.01	Base	0.556
0.8	10	0.10	Base	0.590
0.7	15	0.05	+ clinical skip	0.573
0.7	25	0.05	+ clinical skip	0.590
0.7	**15**	**0.05**	**Base (selected)**	**0.598**

5.8 WEB APPLICATION RESULTS

The trained system is deployed as a local web application at `http://127.0.0.1:5000`. All screens shown below were captured from the running application.

5.8.1 Overview Page

The Overview page (Fig 5.4) introduces the problem, summarises the dataset (100 patients, 378 visit pairs, 1,000 probes), shows TAGT's validated AUC, explains the three stages of the model and displays the research-prototype disclaimer.

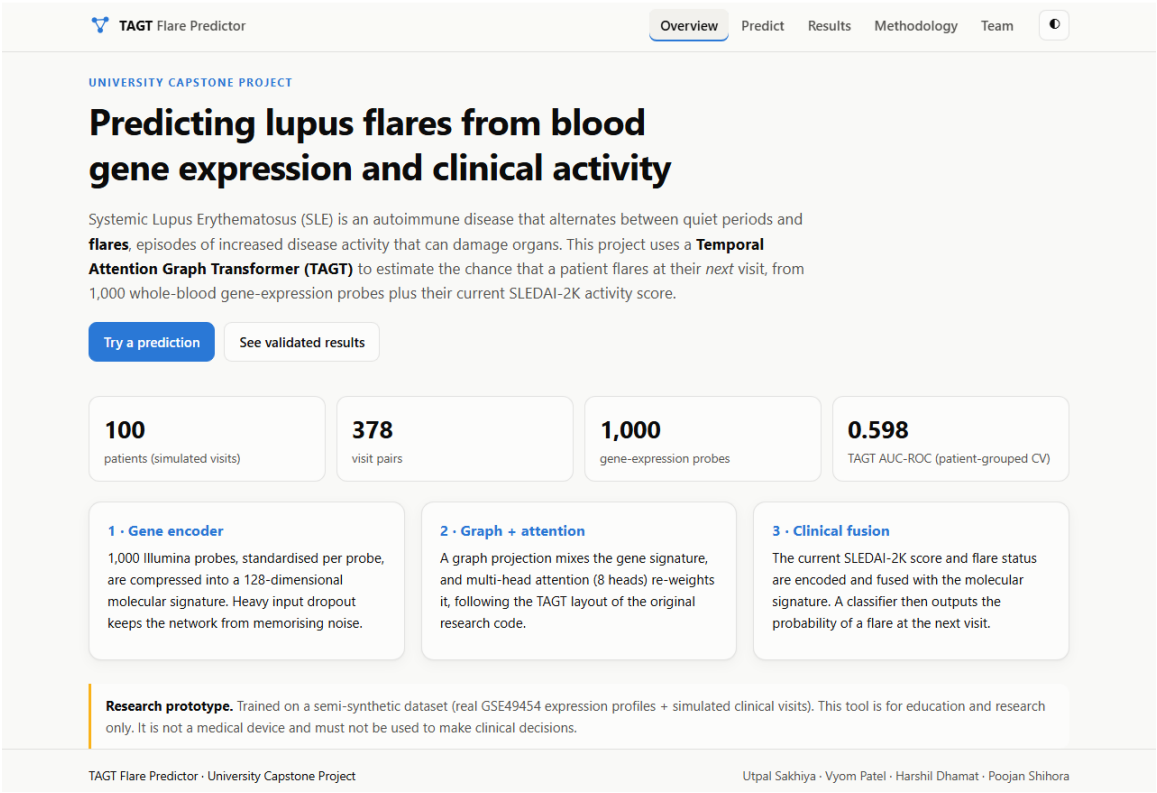


Fig 5.4 Overview Page of the Web Application

5.8.2 Prediction for a Dataset Visit

In cohort-sample mode (Fig 5.5), the user selects one of the 378 visit pairs. The example shows patient PATIENT_0, visit 5 → 6, with a current SLEDAI-2K of 17.2. TAGT predicts a flare probability of 65.6%, above its threshold of 59%, so the risk is High. The clinical baseline gives 37.6%, just below its own threshold (Moderate). After prediction, the true outcome is revealed (flare) and the interface confirms that TAGT was correct. The attribution chart lists the ten probes that most influenced the prediction, in red for probes that raised the risk and blue for probes that lowered it.

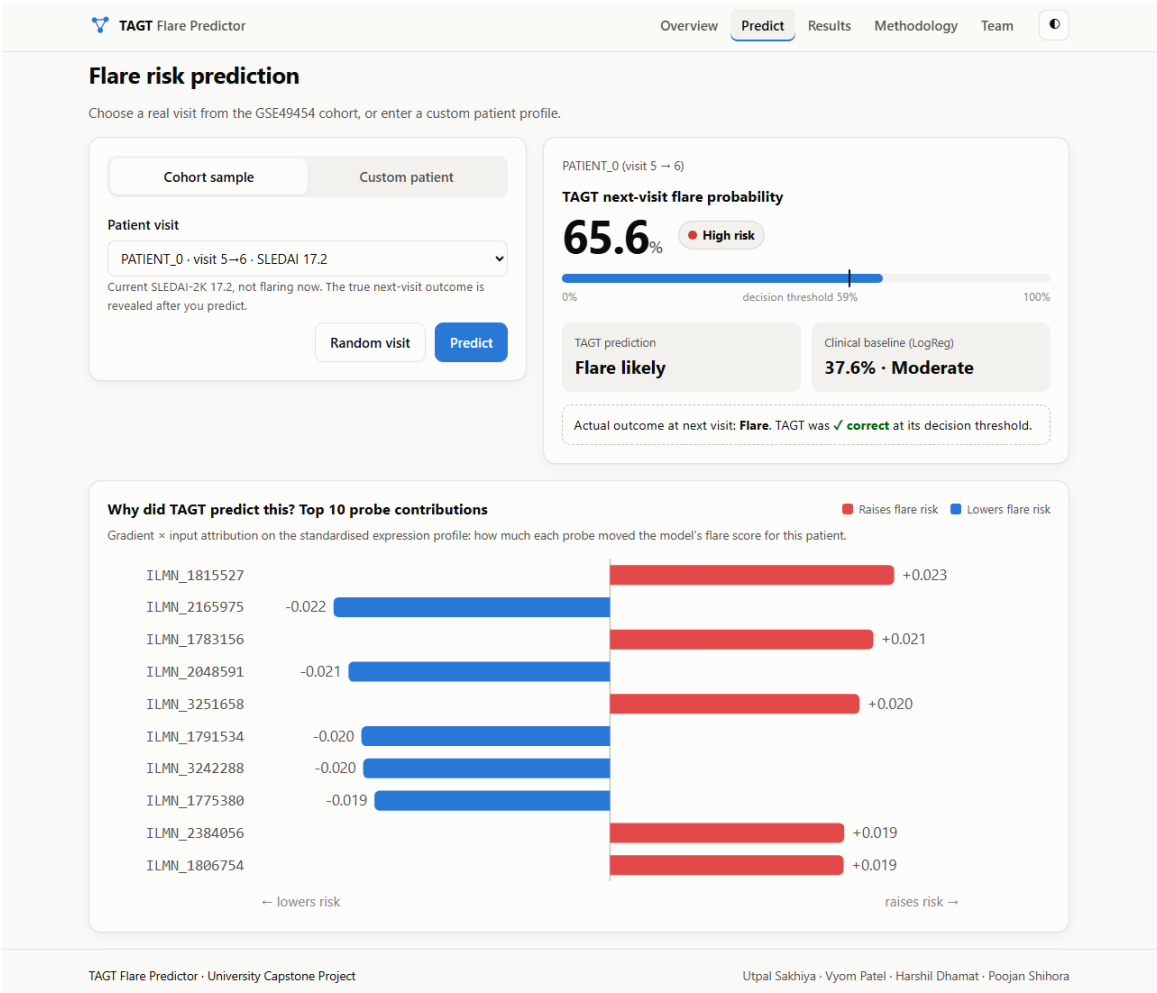


Fig 5.5 Prediction and Explanation for a Dataset Visit

5.8.3 Prediction for a Custom Patient

In custom mode (Fig 5.6), the user sets the SLEDAI-2K score with a slider, indicates whether the patient is currently flaring and may upload an expression CSV. The example uses SLEDAI-2K = 22 with a current flare and no expression file. TAGT predicts 76.5% (High) and the baseline 50.3% (High). Because no expression data were supplied, the interface explains that no gene-level attributions exist and that the prediction is based on the clinical inputs.

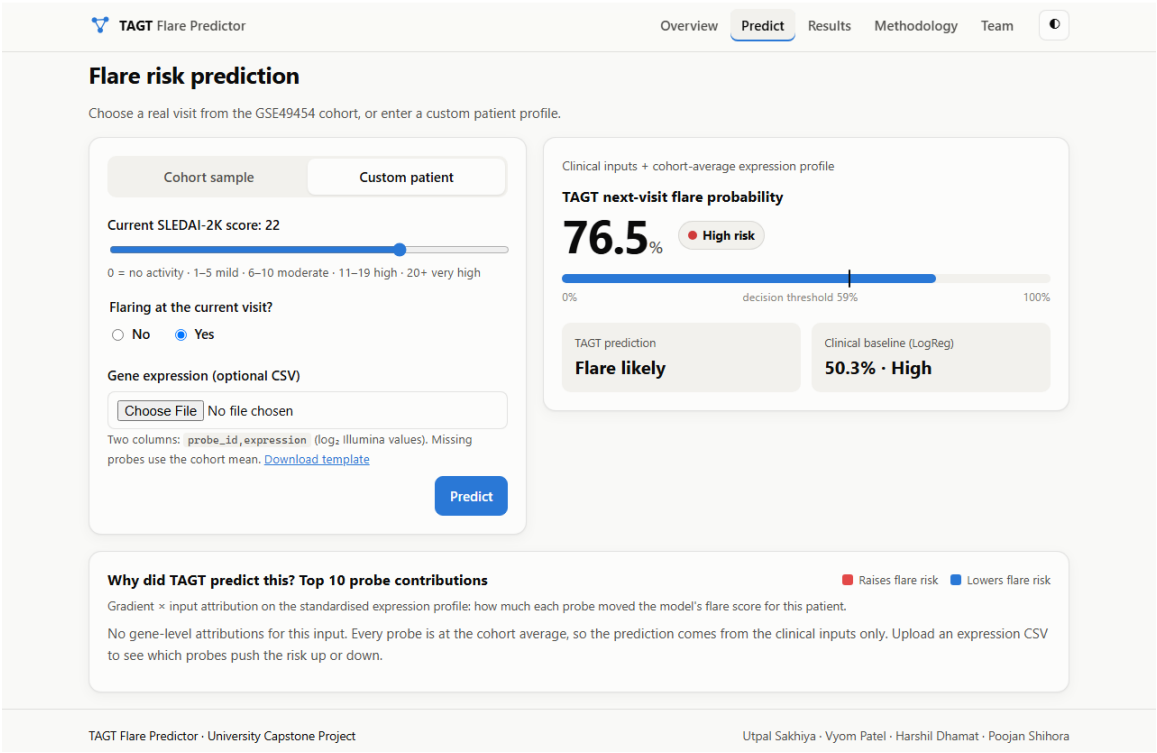


Fig 5.6 Prediction for a Custom Patient Profile

5.8.4 Results, Methodology and Team Pages

The Results page (Fig 5.7) presents interactive ROC curves, the AUC comparison, the full metrics table and automatically generated key findings, all loaded from the metrics file produced by the training pipeline. The Methodology page (Fig 5.8) documents the dataset provenance, inputs, training settings, validation protocol, baselines and limitations. The Team page (Fig 5.9) lists the project members and their enrollment numbers.

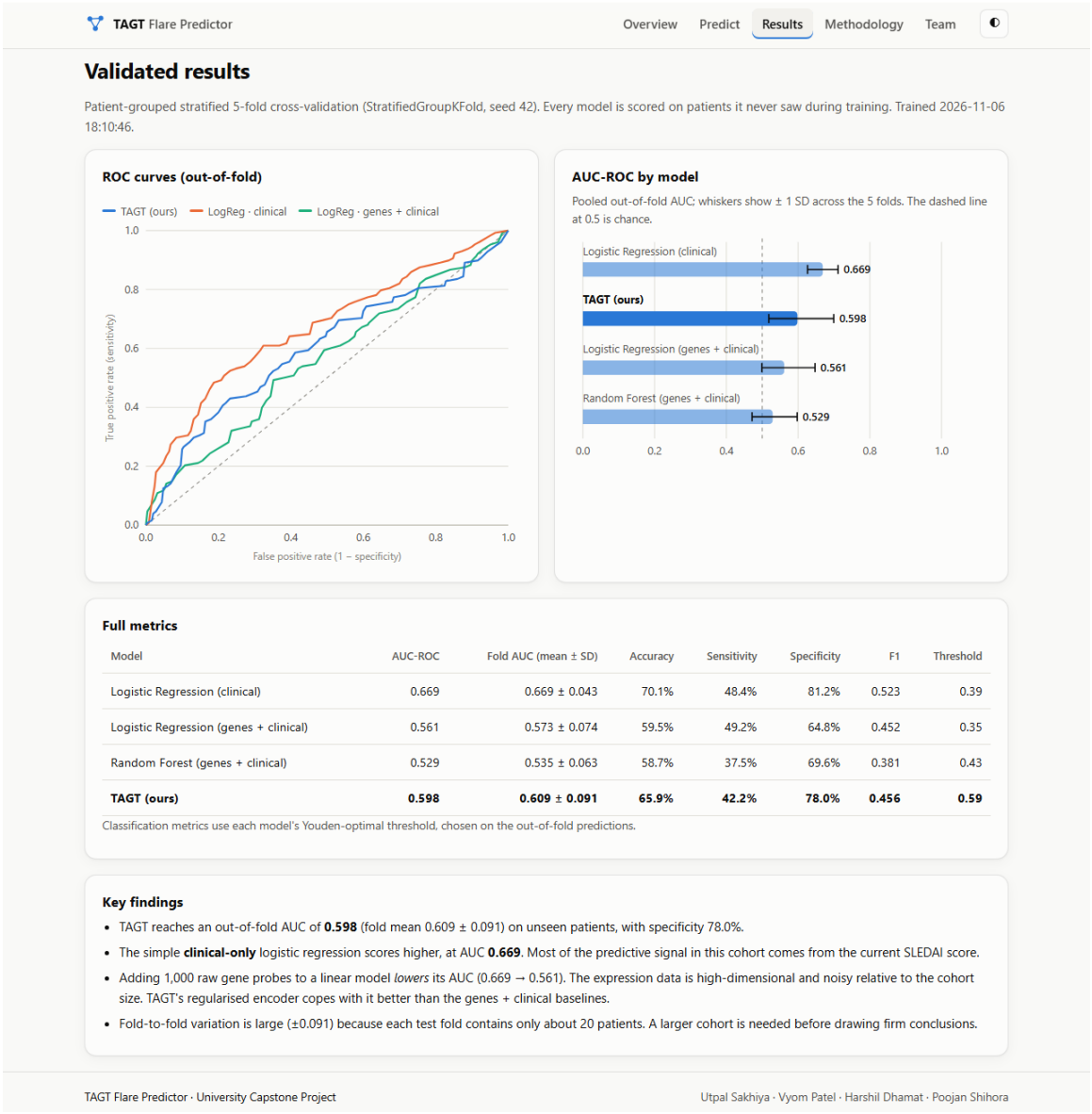


Fig 5.7 Results Page with ROC Curves, AUC Comparison and Metrics

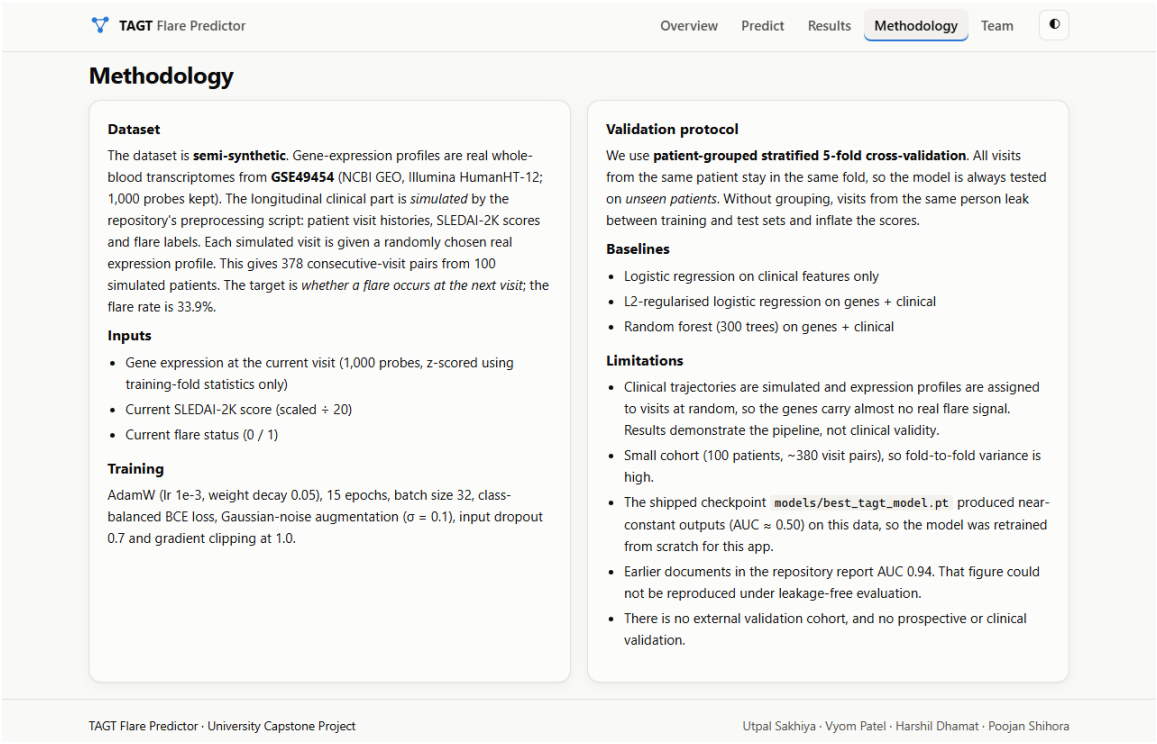


Fig 5.8 Methodology Page

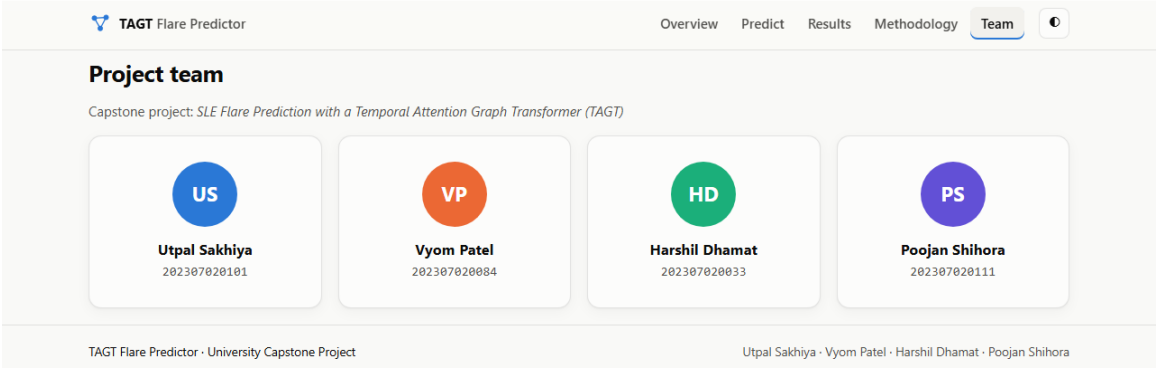


Fig 5.9 Team Page

5.9 COMPARISON WITH PREVIOUSLY REPORTED RESULTS

Documents in the inherited code base report substantially higher performance than this project measured. Table 5.4 contrasts the earlier claims with the reproduced results. The difference is explained by three factors identified in this project: the distributed checkpoint does not reproduce the claimed behaviour, the earlier evaluations did not separate patients between training and test folds, and the dataset's gene-expression profiles are assigned to visits at random.

Table 5.4 Previously Claimed versus Reproduced Performance

Item	Earlier claim	This project (leakage-free)
TAGT AUC-ROC	0.943 ± 0.018	0.598 (folds 0.609 ± 0.091)
TAGT accuracy	89.2%	65.9%
Distributed checkpoint	Best model	Near-constant output, AUC ≈ 0.50
Best model overall	TAGT	Clinical logistic regression (0.669)
Validation protocol	Record-level CV	Patient-grouped stratified 5-fold CV

5.10 DISCUSSION

The results are consistent with how the dataset was constructed. In the simulation, the flare probability depends on the SLEDAI-2K score, and each visit receives a randomly chosen expression profile. The only systematic link between expression and outcome is a small amount of noise whose magnitude grows with SLEDAI-2K. A sound evaluation should therefore find that clinical variables carry most of the signal and that gene probes are close to uninformative. This is exactly what was observed. A logistic regression on genes alone (without clinical features) reached an AUC of only 0.511 under the same protocol, and adding the 1,000 probes to the clinical model *reduced* its AUC from 0.669 to 0.561.

In this setting, TAGT's advantage over the gene-based baselines shows that its regularised encoder handles 1,000 noisy features better than linear or tree-based models. It learns to rely mainly on its clinical branch instead of fitting spurious gene patterns. TAGT still does not beat the clinical-only model, which shows that model complexity cannot create signal that is absent from the data. Equally important, the leakage-free protocol and the comparison with simple baselines exposed this situation, whereas the earlier evaluation had produced an apparently excellent but misleading result.

From an engineering perspective, the project delivers what it set out to build. The pipeline is reproducible: one command regenerates all models and metrics in under 20 seconds. The API returns predictions with explanations in well under a second on a CPU. The interface makes both predictions and their limitations understandable to non-specialists. The same pipeline can be retrained without code changes on a genuinely longitudinal cohort with matched expression and clinical data.

5.11 LIMITATIONS AND THREATS TO VALIDITY

- **Semi-synthetic data:** clinical trajectories are simulated and expression profiles are assigned at random, so the gene-level findings and attributions have no biological meaning, and the absolute performance figures do not transfer to real patients.
- **Small sample:** 100 patients and 378 visit pairs lead to wide fold-to-fold variation (TAGT AUC from 0.446 to 0.713).
- **Selection on evaluation folds:** the TAGT regularisation settings were chosen on the same folds used for reporting, so the selected configuration's AUC is slightly optimistic. A nested cross-validation would remove this bias.
- **Single time step:** each prediction uses only the current visit, so the temporal capability of the attention block is not exploited.
- **No external validation:** no independent cohort was available for testing.

5.12 SUMMARY

Under leakage-free validation, TAGT achieved an AUC-ROC of 0.598 and outperformed the gene-based baselines, while the clinical-only baseline performed best overall (0.669). These findings are explained by the semi-synthetic construction of the dataset. The deployed application presents predictions, explanations and validated metrics transparently. The final chapter summarises the contributions and outlines future work.

CHAPTER 6

CONCLUSION AND FUTURE WORK

6.1 CONCLUSION

This project designed, implemented, validated and deployed the TAGT Flare Predictor, a system that estimates the probability of an SLE flare at the next clinical visit from whole-blood gene expression and the current SLEDAI-2K score. The work turned an inherited research code base, whose trained model was non-functional and whose headline claims could not be verified, into a reproducible, honestly evaluated and usable application.

6.2 KEY OUTCOMES

- A working TAGT implementation in PyTorch (400,737 parameters) with regularisation suited to high-dimensional, small-sample data.
- A leakage-free evaluation pipeline using patient-grouped stratified 5-fold cross-validation, with three baselines evaluated on identical folds.
- Validated results: TAGT AUC-ROC 0.598, better than the gene-based baselines (0.561 and 0.529); clinical-only baseline 0.669.
- An audit showing that the distributed checkpoint was non-functional (AUC ≈ 0.50) and that the earlier claim of 0.943 AUC is not reproducible.
- Identification and disclosure of the semi-synthetic nature of the dataset, which explains the observed results.
- Per-prediction explanations based on gradient × input attributions.
- A Flask REST API and an offline-capable, responsive web interface with prediction, explanation, results and methodology pages, launched with a single script.

6.3 LIMITATIONS

The principal limitation is the semi-synthetic dataset. Because expression profiles are not genuinely linked to the clinical outcome, the project cannot establish whether blood transcriptomics improves flare prediction in real patients. The small number of patients, the single-time-step formulation and the absence of external validation further limit generalisation, as discussed in Section 5.11.

6.4 FUTURE WORK

1. **Real longitudinal data:** retrain and evaluate the unchanged pipeline on a cohort with matched visit-level expression and clinical records, such as the full GSE49454 sample annotations or the longitudinal cohort of Banchereau et al. [1].
2. **True temporal modelling:** feed sequences of several past visits to the attention block, so that TAGT can learn from trajectories rather than a single snapshot.
3. **Biological graph structure:** replace the dense graph projection with message passing over a STRING protein–protein interaction network [23] restricted to the measured genes.
4. **Gene-module features:** aggregate probes into interferon and other transcriptional modules [4] to reduce dimensionality and improve interpretability.
5. **Nested cross-validation and calibration:** tune hyper-parameters in an inner loop and calibrate predicted probabilities.
6. **External validation and deployment:** validate on an independent cohort and package the application in a container for secure hospital deployment.

6.5 CLOSING REMARKS

The most valuable lesson of this project is that rigorous validation matters as much as model architecture. A sophisticated model evaluated carelessly can appear excellent while learning little. A simple clinical baseline and a patient-grouped protocol revealed what the data could and could not support. The resulting system is transparent about its own limits, and it provides a solid, reusable foundation for future work on real SLE cohorts.

APPENDIX 1

INSTALLATION AND USER GUIDE

(TAGT Flare Predictor: SLE Flare Prediction using a Temporal Attention Graph Transformer)

INSTALLATION ON WINDOWS

1. Install Python 3.10 or later and ensure that `python` is available on the PATH.
2. Open the project folder and double-click `run_webapp.bat`.
3. On the first run, the script creates a virtual environment, installs CPU PyTorch and the other dependencies, and then starts the server. This takes a few minutes.
4. The browser opens `http://127.0.0.1:5000` automatically. Press Ctrl + C in the console window to stop the server.

MANUAL INSTALLATION (ANY OPERATING SYSTEM)

```
python -m venv .venv
# Windows: .venv\Scripts\activate    Linux/macOS: source .venv/bin/activate
pip install torch --index-url https://download.pytorch.org/whl/cpu
pip install -r requirements-webapp.txt
python webapp/train.py     # optional: retrain all models (~20 s)
python webapp/app.py       # then open http://127.0.0.1:5000
```

USING THE APPLICATION

- **Predict → Cohort sample:** choose a visit pair (or press Random visit) and press Predict. The true next-visit outcome is shown after the prediction.
- **Predict → Custom patient:** set the SLEDAI-2K score and current flare status. Optionally upload a CSV with columns `probe_id,expression` (download the template from the same page).
- **Results:** hover over the ROC curves and bars for exact values.
- **Theme:** use the ◐ button in the top bar to switch between light and dark mode.

APPENDIX 2

CORE SOURCE CODE

(TAGT Flare Predictor: SLE Flare Prediction using a Temporal Attention Graph Transformer)

TAGT MODEL (webapp/tagt_core.py)

```
class TAGTModel(nn.Module):
    def __init__(self, n_genes=1000, clinical_dim=2, dropout=0.3, input_dropout=0.7):
        super().__init__()
        self.gene_encoder = nn.Sequential(
            nn.Dropout(input_dropout),
            nn.Linear(n_genes, 256), nn.ReLU(), nn.Dropout(dropout), nn.Linear(256,
128))
        self.graph_conv = nn.Linear(128, 128)
        self.temporal_attention = nn.MultiheadAttention(128, num_heads=8,
batch_first=True)
        self.clinical_encoder = nn.Sequential(nn.Linear(clinical_dim, 32), nn.ReLU())
        self.classifier = nn.Sequential(
            nn.Linear(160, 128), nn.ReLU(), nn.Dropout(dropout),
            nn.Linear(128, 64), nn.ReLU(), nn.Dropout(dropout),
            nn.Linear(64, 1))

    def forward(self, genes, clinical):
        g = torch.relu(self.graph_conv(self.gene_encoder(genes))).unsqueeze(1)
        attn, _ = self.temporal_attention(g, g, g)
        fused = torch.cat([attn.squeeze(1), self.clinical_encoder(clinical)], dim=1)
        return self.classifier(fused).squeeze(-1)
```

PATIENT-GROUPED CROSS-VALIDATION (webapp/train.py)

```
cv = StratifiedGroupKFold(n_splits=5, shuffle=True, random_state=42)
for k, (tr, te) in enumerate(cv.split(X, y, groups)):
    pre = Preprocessor().fit(X[tr])                # statistics from training fold
only
    model = train_tagt(pre.genes(X[tr]), Cs[tr], y[tr], seed=42 + k)
    oof['TAGT (ours)'][te] = tagt_predict(model, pre.genes(X[te]), Cs[te])
    for name, make in baselines.items():
        clf = make().fit(feats[name](X[tr], Cs[tr]), y[tr])
        oof[name][te] = clf.predict_proba(feats[name](X[te], Cs[te]))[:, 1]
```

PREDICTION WITH EXPLANATION (webapp/app.py)

```
def predict_one(expression, sledai, flare_now):
    clin = Preprocessor.clinical(np.array([[sledai, flare_now]], np.float32))
    genes = torch.tensor(PRE.genes(expression[None, :]), requires_grad=True)
    logit = MODEL(genes, torch.tensor(clin))
    prob = float(torch.sigmoid(logit).item())
    logit.sum().backward()
    contrib = (genes.grad * genes).detach().numpy()[0]   # gradient x input
    top = np.argsort(-np.abs(contrib))[:10]
    ...
```

REFERENCES

1. Banchereau, R., Hong, S., Cantarel, B., Baldwin, N., Baisch, J., Edens, M., Cepika, A.M., Acs, P., Turner, J., Anguiano, E., Vinod, P., Kahn, S., Obermoser, G., Blankenship, D., Wakeland, E., Nassi, L., Gotte, A., Punaro, M., Liu, Y.J., Banchereau, J., Rossello-Urgell, J., Wright, T. and Pascual, V. (2016) 'Personalized Immunomonitoring Uncovers Molecular Networks that Stratify Lupus Patients', Cell, Vol.165, No.3, pp.551-565.
2. Barrett, T., Wilhite, S.E., Ledoux, P., Evangelista, C., Kim, I.F., Tomashevsky, M., Marshall, K.A., Phillippy, K.H., Sherman, P.M., Holko, M., Yefanov, A., Lee, H., Zhang, N., Robertson, C.L., Serova, N., Davis, S. and Soboleva, A. (2013) 'NCBI GEO: Archive for Functional Genomics Data Sets – Update', Nucleic Acids Research, Vol.41, No.D1, pp.D991-D995.
3. Breiman, L. (2001) 'Random Forests', Machine Learning, Vol.45, No.1, pp.5-32.
4. Chiche, L., Jourde-Chiche, N., Whalen, E., Presnell, S., Gersuk, V., Dang, K., Anguiano, E., Quinn, C., Burtey, S., Berland, Y., Kaplanski, G., Harle, J.R., Pascual, V. and Chaussabel, D. (2014) 'Modular Transcriptional Repertoire Analyses of Adults with Systemic Lupus Erythematosus Reveal Distinct Type I and Type II Interferon Signatures', Arthritis & Rheumatology, Vol.66, No.6, pp.1583-1595.
5. Collins, G.S., Reitsma, J.B., Altman, D.G. and Moons, K.G.M. (2015) 'Transparent Reporting of a Multivariable Prediction Model for Individual Prognosis or Diagnosis (TRIPOD): The TRIPOD Statement', BMJ, Vol.350, g7594.
6. Fawcett, T. (2006) 'An Introduction to ROC Analysis', Pattern Recognition Letters, Vol.27, No.8, pp.861-874.
7. Gladman, D.D., Ibañez, D. and Urowitz, M.B. (2002) 'Systemic Lupus Erythematosus Disease Activity Index 2000', The Journal of Rheumatology, Vol.29, No.2, pp.288-291.
8. Hochreiter, S. and Schmidhuber, J. (1997) 'Long Short-Term Memory', Neural Computation, Vol.9, No.8, pp.1735-1780.
9. Kaufman, S., Rosset, S., Perlich, C. and Stitelman, O. (2012) 'Leakage in Data Mining: Formulation, Detection, and Avoidance', ACM Transactions on Knowledge Discovery from Data, Vol.6, No.4, Article 15, pp.1-21.
10. Kaul, A., Gordon, C., Crow, M.K., Touma, Z., Urowitz, M.B., van Vollenhoven, R., Ruiz-Irastorza, G. and Hughes, G. (2016) 'Systemic Lupus Erythematosus', Nature Reviews Disease Primers, Vol.2, Article 16039.
11. Kipf, T.N. and Welling, M. (2017) 'Semi-Supervised Classification with Graph Convolutional Networks', Proc. 5th International Conference on Learning Representations (ICLR), Toulon, France.

12. Loshchilov, I. and Hutter, F. (2019) ‘Decoupled Weight Decay Regularization’, Proc. 7th International Conference on Learning Representations (ICLR), New Orleans, LA.

13. National Center for Biotechnology Information (2014) ‘GEO Accession Viewer: Series GSE49454’, available at https://www.ncbi.nlm.nih.gov/geo/query/acc.cgi?acc=GSE49454 (accessed November 2026).

14. Pallets Projects (2026) ‘Flask Documentation (Stable)’, available at https://flask.palletsprojects.com/en/stable/ (accessed November 2026).

15. Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Köpf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., Bai, J. and Chintala, S. (2019) ‘PyTorch: An Imperative Style, High-Performance Deep Learning Library’, Advances in Neural Information Processing Systems, Vol.32, pp.8024-8035.

16. Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M. and Duchesnay, E. (2011) ‘Scikit-learn: Machine Learning in Python’, Journal of Machine Learning Research, Vol.12, pp.2825-2830.

17. Petri, M., Orbai, A.M., Alarcón, G.S., Gordon, C., Merrill, J.T., Fortin, P.R., Bruce, I.N., Isenberg, D., Wallace, D.J., Nived, O., Sturfelt, G., Ramsey-Goldman, R., Bae, S.C., Hanly, J.G., Sánchez-Guerrero, J., Clarke, A., Aranow, C., Manzi, S., Urowitz, M., Gladman, D., Kalunian, K., Costner, M., Werth, V.P., Zoma, A., Bernatsky, S., Ruiz-Irastorza, G., Khamashta, M.A., Jacobsen, S., Buyon, J.P., Maddison, P., Dooley, M.A., van Vollenhoven, R.F., Ginzler, E., Stoll, T., Peschken, C., Jorizzo, J.L., Callen, J.P., Lim, S.S., Fessler, B.J., Inanc, M., Kamen, D.L., Rahman, A., Steinsson, K., Franks, A.G., Sigler, L., Hameed, S., Fang, H., Pham, N., Brey, R., Weisman, M.H., McGwin, G. and Magder, L.S. (2012) ‘Derivation and Validation of the Systemic Lupus International Collaborating Clinics Classification Criteria for Systemic Lupus Erythematosus’, Arthritis & Rheumatism, Vol.64, No.8, pp.2677-2686.

18. Rajkomar, A., Dean, J. and Kohane, I. (2019) ‘Machine Learning in Medicine’, New England Journal of Medicine, Vol.380, No.14, pp.1347-1358.

19. scikit-learn developers (2026) ‘sklearn.model_selection.StratifiedGroupKFold’, scikit-learn API Reference, available at https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.StratifiedGroupKFold.html (accessed November 2026).

20. Shrikumar, A., Greenside, P. and Kundaje, A. (2017) ‘Learning Important Features Through Propagating Activation Differences’, Proc. 34th International Conference on Machine Learning (ICML), PMLR Vol.70, pp.3145-3153.

21. Simonyan, K., Vedaldi, A. and Zisserman, A. (2014) ‘Deep Inside Convolutional Networks: Visualising Image Classification Models and Saliency Maps’, Workshop Proc. 2nd International Conference on Learning Representations (ICLR), Banff, Canada.

22. Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I. and Salakhutdinov, R. (2014) 'Dropout: A Simple Way to Prevent Neural Networks from Overfitting', Journal of Machine Learning Research, Vol.15, No.56, pp.1929-1958.

23. Szklarczyk, D., Gable, A.L., Lyon, D., Junge, A., Wyder, S., Huerta-Cepas, J., Simonovic, M., Doncheva, N.T., Morris, J.H., Bork, P., Jensen, L.J. and von Mering, C. (2019) 'STRING v11: Protein–Protein Association Networks with Increased Coverage, Supporting Functional Discovery in Genome-Wide Experimental Datasets', Nucleic Acids Research, Vol.47, No.D1, pp.D607-D613.

24. Tsokos, G.C. (2011) 'Systemic Lupus Erythematosus', New England Journal of Medicine, Vol.365, No.22, pp.2110-2121.

25. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, Ł. and Polosukhin, I. (2017) 'Attention Is All You Need', Advances in Neural Information Processing Systems, Vol.30, pp.5998-6008.

26. Veličković, P., Cucurull, G., Casanova, A., Romero, A., Liò, P. and Bengio, Y. (2018) 'Graph Attention Networks', Proc. 6th International Conference on Learning Representations (ICLR), Vancouver, Canada.

27. Youden, W.J. (1950) 'Index for Rating Diagnostic Tests', Cancer, Vol.3, No.1, pp.32-35.